

Atlantis — architectural north star

The contract every vocabulary writes against, and the principles that keep the constellation coherent. Companion to [plan.md](#) (priorities), [vetula-design.md](#) (one worked vocabulary), and [gaps.md](#) (claim-vs-code). Authoritative when it contradicts the others.

Authored: 2026-05-22.

1. The animating constraint

Tidal mini-notation is the only parsed sub-language. Everything else is PureScript eDSL.

Mini-notation earns its parser: compression-by-syntax (`bd*4(3,8), <a b c>`) beats the cost of tokenising. Nothing else earns one. A parser per vocabulary would fragment the language, prevent values from flowing between vocabularies, and lock cell text into a UI artefact instead of a typed expression.

Concretely:

- A Vetula declaration is a PureScript record, not a `vetula\n in: cMajor\n ...` mini-grammar.
- An Odonus position array is `[0, 2, 4, 3, 2, 0] :: Array Int`, not a comma-separated string.
- A Selene config is a record-of-records, not a YAML fragment.
- A cell body is a PureScript expression — possibly multi-statement via `let ... in ...` — that evaluates to a `Voice` or `Array Voice`.

This is the architectural constraint that buys everything else.

2. The two typeclasses

Two complementary typeclasses span the substrate:

```
-- Source side. Anything that yields a Pattern is a Notation.
class Notation n a | n -> a where
  toPattern :: n -> Pattern a

-- Sink side. Anything that can consume an event is Emitable.
class Emitable d where
  emit :: d -> Event ValueMap -> Effect Unit
```

The substrate sits between, and has two responsibilities:

1. **Run** — for each voice, query its `Notation` and hand each event to its `Emitable`.
2. **Publish** — expose each voice's output record to the live- control bus under its voice name, so sibling voices in *other* cell-modules can read it (see §5).

```

runVoice :: forall n a d
  . Notation n a => Emitable d
  => n -> d -> Window -> Effect Unit
runVoice src dst window =
  traverse_ (emit dst) (queryWindow window (toPattern src))

publishVoiceOutput :: forall o
  . VoiceOutput o
  => String -> o -> Effect Unit
publishVoiceOutput voiceName output =
  for_ (outputSlots output) \(Tuple slot value) ->
    busPut (voiceName <> "." <> slot) value

```

The publish half is what lets the per-cell-module reality (each twice → its own compiled PureScript module) not break the picture in §3. See §5 for why this is load-bearing.

Notation instances we expect:

Instance	Yields
MiniNotation a	Pattern a
Vetula	Pattern Voicing
Balistes	Pattern (Array DrumHit)
Odonus	Pattern Pitch
Selene	Pattern OctoSignal
Pattern a (self-instance)	Pattern a

Emitable instances we expect:

Instance	Wire format
SuperDirt	/dirt/play OSC bundles to localhost:57120
MidiPort	Note-On/Off + CCs to a CoreMIDI destination
MidiDrumKit	Note-On per slot, with sound→slot map
CvRouter	V/oct + Gate via OSC to cv-router daemon
Fh2Daemon	SysEx writes for selene config

This is the whole architecture in two lines: vocabularies are **Notation**, sinks are **Emitable**, the substrate composes them.

3. Cells are PureScript expressions

A cell evaluates to a **Voice** or **Array Voice**. A **Voice** binds a **Notation** to an **Emitable**.

```

-- A Voice is an opaque pair of (notation, destination) – both
-- existentially quantified so the substrate can hold heterogeneous
-- voices in one Array.
data Voice = forall n a d
    . (Notation n a, Emitable d)
    => Voice n d

(>>) :: forall n a d
    . (Notation n a, Emitable d)
    => n -> d -> Voice
src >> dst = Voice src dst

```

Intra-cell — lexical scope within one compilation unit

Within a single cell, normal PureScript lexical scope applies. This is useful when several values are tightly coupled and the user wants them moved as a group:

```

let chord1 = vetula { in: cMajor
                      , progression: mcmullenYellow
                      , voicing: pianoStyle
                      , octave: 4
                    }

    melody = odonusOver chord1 [0, 2, 4, 3, 2, 0]
    drums  = mini "bd sn hh cp" :: MiniNotation Sample
in
  [ melody >> piano1
  , (s drums # gain (lfo chord1.rms 0.3 1.0)) >> sample -- MIDI piano
  , drums >> superdirt -- modular drums
  , chord1 >> piano1 -- and also SD
  ] -- chord1 emits
too

```

Observations:

- `chord1` is bound once and read three ways: as a `Notation` source (its voicings flow to `piano1`), as a signal source (`chord1.rms` modulates drum gain), and as a structural source (`odonusOver chord1` consults its current voicing).
- `mini "bd sn hh cp"` is a value, not a string. `MiniNotation Sample` is a real type, with `Show`, `Semigroup`, and `Notation` instances.
- `>>` makes the destination explicit at every voice. Today's implicit-SuperDirt behaviour becomes a default in cell sugar; the underlying expression is always `notation >> destination`.

Inter-cell — the multi-module reality

The intra-cell example above compiles to one PureScript module and hot-loads as one unit. But per-cell hot-load — the affordance that makes live coding work — requires each twice to be its OWN module: edits to `melody` should not force a recompile of `chord1`.

Once cells are separate modules, **PureScript lexical scope no longer spans them**. Cell B cannot reach into Cell A's `let`-bindings. The intra-cell example above must be unrolled to one cell per voice, with references between them mediated by the live-control bus. See §5 for the mechanism in detail — `chord1`'s voice publishes a `VetulaOutput` record under its voice name; `melody`'s cell reads it via `live`-style typed bus reads.

The intra-cell shape stays useful (tightly-coupled local values move together; type-check tells you a stale reference); the inter-cell shape is the one that survives hot-load across the session.

4. Verb-vs-sink: consistent shape, not consistent implementation

The same vocabulary flows to every sink. Each sink consumes what it understands and ignores the rest. Holes are fine; they fall out honestly as the table makes them visible.

Each cell of the table is one of:

- **D** — direct: sink natively understands this verb
- **M** — mappable: translated via per-rig config (e.g. soft-synth CC table)
- **—** — no-op: silently dropped
- **G** — global: affects the engine, not one voice

A representative slice (full table in code, regenerated when verbs or sinks change):

	SuperDirt	MIDI generic	MIDI drumkit	FH-2	CvRouter	Yarns poly
<code>note</code>	D	D	—	—	D	D
<code>n</code>	D	—	—	—	—	—
<code>sound</code>	D	—	D (slot)	—	—	—
<code>gain</code>	D	D (→ vel)	D (→ vel)	—	D (→ CV)	D (→ vel)
<code>pan</code>	D	M (CC10)	M (CC10)	—	M	M (CC10)
<code>velocity</code>	D	D	D	—	M	D
<code>legato</code>	D	D (note-len)	—	—	D (gate)	D (gate)
<code>lpf</code>	D	M (CC74)	—	—	M	M (CC74)
<code>room</code>	D	M (CC91)	—	—	—	M (CC91)
<code>attack</code>	D	M (CC73)	—	—	—	M (CC73)
<code>crush</code>	D	—	—	—	—	—
<code>vowel</code>	D	—	—	—	—	—
<code>coarse</code>	D	M (PB)	—	—	D (CV+)	M (pgm)
<code>cut</code>	D	M (note-off)	D (group)	—	D (gate-)	D
<code>nudge</code>	D	D (schedule)	D (schedule)	D	D	D

	SuperDirt	MIDI generic	MIDI drumkit	FH-2	CvRouter	Yarns poly
cps	G	G	G	G	G	G
octo-signal	—	—	—	D	—	—
chord/voicing	(via N)	(via N)	—	—	(via N)	D

The **M-cells are the per-rig translation work**. They live in a Studio-level declaration that says "this Ableton instrument: lpf → CC74, room → CC91, attack → CC73". A modular voice's M-cells say "lpf → aux CV jack 3 on ES-9 bank A". The translation table is hardware/patch-specific data, not code.

The **dashes are honest holes** that fall out for free. A cell that writes `# vowel "a"` against a MIDI drum kit silently ignores the vowel. That's correct. We don't fight to make it work.

5. Cross-cell references — the mediating layer

This is the place where the "all PureScript, all composable" story admits a layer. Per-cell hot-load — what makes the live-coding experience work — requires that each twice be its OWN PureScript module, compiled and reloaded independently. Once cells live in separate modules, PureScript lexical scope cannot reach across them. A cell that wants to read another cell's output needs a runtime mechanism, not a compile-time one.

That runtime mechanism is the **live-control bus + voice-output records**. It is load-bearing: every cross-cell reference goes through it. Two scopes therefore co-exist in the architecture:

Intra-cell — PureScript lexical scope

Within one cell (one compilation unit), `let`-bindings work normally, type-check normally, and edits move as a group. Use this when values are tightly coupled and you want the type system to catch broken references when you edit one of them.

Inter-cell — the bus

Cell B reading from cell A goes through the live-control bus. The bus is typed by convention: each running voice publishes an output record under its voice name, and other cells read from it via a typed `live` family. Vetula's output, by convention:

```
type VetulaOutput =
  { voicings      :: Pattern Voicing    -- the Notation payload
  , currentVoicing :: Voicing           -- queryable now
  , rootPitch     :: Pattern Pitch      -- derived signal
  , rms           :: Pattern Number     -- derived signal (chord density)
  , degree        :: Pattern Numeral   -- derived signal
  }
```

Every vocabulary publishes a similar record. `Odonus` publishes its current pitch and position; `Balistes` its current step and density; `Selene` its current modulator values. The field names become the bus slot names: `chord1.currentVoicing`, `chord1.rms`, `bass1.currentPitch`, etc.

Typed bus reads

The bus today returns **Pattern Number** from **live "slotname"**. The architecture generalises this to per-type reads:

```
class LiveReadable a where
  live :: String -> Pattern a

instance LiveReadable Number where ...
instance LiveReadable Voicing where ...
instance LiveReadable Pitch where ...
```

So **live "chord1.currentVoicing" :: Pattern Voicing** works directly, and **odonusOver** becomes a function that takes a **Pattern Voicing** plus a position array.

Twister-style runtime sources continue to push to **Number** slots; AI agents and sibling voices can push to richer slots when the publication makes them readable.

The honest unrolling of §3

The intra-cell example in §3 unrolls to four cells across cell-module boundaries:

```
-- Cell `chord1` (one module, hot-loaded independently)
chord1 = vetula { in: cMajor
                  , progression: mcmullenYellow
                  , voicing: pianoStyle
                  , octave: live "chord1.octave" -- live-controllable
                }
in chord1 >> piano1

-- Cell `melody` (separate module – references chord1 via the bus)
melody = odonusOver (live "chord1.currentVoicing") [0, 2, 4, 3, 2, 0]
in melody >> piano2

-- Cell `drums` (separate module)
drums = mini "bd sn hh cp" :: MiniNotation Sample
in (s drums # gain (range 0.3 1.0 (live "chord1.rms"))) >> rample

-- Cell `aux` – also emits chord1 as samples
in (live "chord1.voicings" :: Pattern Voicing) >> superdirt
```

Each cell is a separate module under **Tidal.Generated.M<hash>**. Editing **drums** recompiles only **drums**. **chord1**'s publication is a runtime fact; **melody**'s read is a runtime fact. PureScript types check each cell against the typed bus interface; mismatches surface as compile errors in the consuming cell, not the producing cell.

The price is honest: cross-cell references aren't statically checked end-to-end (a renamed **chord1** doesn't trigger a type error in **melody**). The mitigation: voice-output records are part of the vocabulary's

published API, so the bus slot names are stable, and the typed `live` reads at least catch type mismatches.

What this means for the "fractal" property

The fractal property — patterns of patterns of patterns — survives cleanly:

- **Within one cell:** PureScript composition (functions, `let`, typeclass instances). Type-checked end-to-end.
- **Across cells:** bus references with typed reads. Slot names form a flat namespace, each value typed at the reader.

Both shapes use the same Pattern algebra; the difference is whether the cross-reference travels through the compiler or through the running engine. A piece can mix them freely — a tightly coupled chord+RMS pair inside one cell, then half a dozen other cells reading that pair's outputs by name.

6. Mini-notation as a first-class type

Today: `mini "bd sn" :: Pattern String`. Once parsed, the source is gone.

After: `mini "bd sn" :: MiniNotation Sample`, with

```
newtype MiniNotation a = MiniNotation TPat
  -- TPat is the post-parse tree from Tidal.Parse.Parser

instance Notation (MiniNotation a) a where
  toPattern (MiniNotation tp) = tpatToPattern tp

instance Show (MiniNotation a) where
  show (MiniNotation tp) = renderTPat tp      -- round-trip to source

instance Semigroup (MiniNotation a) where
  append (MiniNotation a) (MiniNotation b) = MiniNotation (catTPat [a, b])
```

What this buys us:

- **Round-trip to source.** Cells can persist the literal source the user typed, not the resolved Pattern. Pretty-print, edit, re-parse.
- **Source-level composition.** `mini "bd sn" <> mini "hh cp"` composes at the tree level, equivalent to `mini "bd sn hh cp"`.
- **Substrate uniformity.** `MiniNotation a` is a `Notation` like any other. The substrate doesn't special-case it.
- **UI affordance.** An editor (Calypso, VS Code) can render the source AND query the resolved Pattern for visualisation without re-parsing — both are in the value.

The same shape applies to other notation values that have meaningful "source" (a Vetula record, an Odonus declaration): the `Show` instance round-trips, the `Notation` instance resolves to Pattern. Cells edit the source; the substrate runs the resolved Pattern.

7. Front-end independence

The cell language IS PureScript. Editors are interchangeable:

- **Calypso** — browser notebook with cells, temporary cards, hot-load, side-by-side panes. Optimised for live performance and rapid iteration.
- **VS Code with PureScript LSP** — file-based, type-checked, version-controlled. Optimised for composition over time and multi-session work.
- **AI agent** — produces PureScript text via the same compile + hot-load pipeline as Calypso. Optimised for unattended improvisation and structural proposals.

All three write the same language to the same engine. Switching between them is a matter of preference and workflow, not capability. This is what the PureScript-eDSL constraint buys us: front-end choice stays open indefinitely.

Calypso's distinctive contribution

Calypso is not "the only way to use the engine". It's one front-end whose distinctive value is the **affordance layer** — things that are hard to express in a file-based editor:

- **Temporary cards with history.** Each cell carries its edit history. Click-to-restore an earlier version without losing the current one. Live-coding's "what was I just doing?" gets a first-class answer. This is a genuine contribution to notebook UX, not just a port of Jupyter.
- **Per-cell hot-load.** Cells compile and reload independently. A typo in cell B doesn't disturb cell A's voice.
- **Live-control bindings.** Twister knob → **live "chord1.octave"** with a per-rig binding map. Editor-side surface for parameter performance.
- **Side-by-side panes.** Composition / arrangement / studio / library views without window-manager juggling.
- **Multi-headed sessions.** Humans + agents collaborating against one BEAM engine; pen-based write-lock; proposal queue.

A VS Code path that wants any of these can implement them on its side of the WebSocket — Calypso doesn't own them by virtue of being the default.

What's portable, what's editor-local

Portable	Editor-local
Cell text (PureScript expr)	Cell layout / pane shape
Library entries (PureScript)	Card history
Voice declarations	Per-rig CC translation tables (in Studio code, but loaded by editor)
Live-control bus protocol	Controller binding UI
WebSocket protocol	Proposal queue UI

The engine is the durable interface. Editors render and edit; they don't define the language.

8. What's already in place vs what this doc points to

Shipped substrate

- Pattern algebra and ControlPattern (# join, mini parser, classic Tidal combinators)
- Per-cell PureScript compile + hot-load
- Live-control bus (set-control / live)
- PortClaim (slot reservation for autonomous emitters)
- Multi-voice supervisor with shared phase
- Several Emitable-shaped sinks in atlantis (MIDI, CvRouter, Fh2Daemon) — but without the typeclass formalised
- Studio/Instrument/Destination declarations (Slab A)

What this doc points to (in dependency order)

1. **Verb-vs-sink table populated** — an artefact in the repo, kept in sync with code, shows D/M/—/G per (verb, sink) cell. Also surfaces in Calypso's Studio pane as a per-destination capability readout.
2. **Notation typeclass formalised** — current vocabulary implementations become instances. Substrate stops caring what it's holding; it calls toPattern.
3. **MiniNotation a as a real type** — Tidal.Cell.Prelude.mini returns this; cells store source and resolved Pattern together; Show/Semigroup/Notation instances; round-trip in editors.
4. **Voice-output records standardised** — Vetula's .rms / .rootPitch shape becomes the convention every vocabulary follows for what it exposes upward.
5. **>> operator + Voice ADT** — destination explicit in cell text; defaultable per voice; the substrate consumes Array Voice.
6. **Per-rig translation layer** — Studio declarations carry the M-cell mappings (this MIDI instrument: lpf → CC74; this modular voice: lpf → aux CV jack B7). Soft-synth and modular sinks gain wide-coverage cells without sink-side code.
7. **Selene as Notation** — current octo-signal-as-notation work (Slab C, task #59) slots in as a Notation Selene Selene instance and a Emitable Fh2Daemon instance.
8. **Multi-cv-router runtime** — task #68 becomes "another CvRouter instance per router process"; trivial once Emitable is a class.
9. **Vetula implementation** — task #134; becomes a Notation Vetula Voicing instance once the type surface is built.
10. **Naming surgery** (#136) — propagated through the typeclass instance declarations and cell-text vocabulary.

The early steps (1–3) are independent and small. Steps 4–6 are the substantial structural work. 7–10 are existing tasks that fold in once the substrate has the typeclass shape.

9. Out of scope (for this doc)

- **Repo split mechanics.** Covered in plan.md; doesn't affect the architecture shape, only its home.
- **Specific cell-text sugar.** Whether >> piano1 or to: piano1 is the preferred surface is a UI question, settled by Calypso's cell-template implementation. The underlying PureScript expression is voice n d.

- **Real-time clock arithmetic.** F-FUTURE / F-LAT / F-LEAD fixes are already shipped substrate; the typeclass shape inherits them for free.
- **Detailed Calypso UI changes.** Once the typeclass + Voice ADT exists, Calypso's Studio pane, cell editor, and library views all gain natural surfaces — but those are downstream work to scope separately.
- **AI-as-pen-holder modes.** The autonomous-improvisation work is separately interesting; this doc is about the substrate that supports it equally regardless of who's writing cells.

10. Tests of the architecture

Once shipped, the following should be true:

1. A cell that says `let chord1 = vetula { ... } in chord1 >> piano1` plays the McMullen Yellow column on an Ableton soft-piano.
2. The same cell with `chord1 >> superdirt` plays it through SuperDirt's piano sample. *No cell text changes other than the destination binding.*
3. A cell `mini "bd sn hh cp" :: MiniNotation Sample` round-trips to source for editor display, AND resolves to a Pattern for emit, AND composes via `<>` at the source level.
4. The verb-vs-sink table generates from code at build time. Adding a new verb to the SuperDirt vocabulary surfaces a new row with D in the SuperDirt column and — in others, prompting per-rig config decisions.
5. The same engine runs cells from Calypso (browser, WebSocket), from VS Code (file watcher + LSP + WebSocket), and from an AI agent (direct WebSocket). Each produces identical events for identical cell text.
6. A new `Emitable` (say, a hypothetical Selene-but-for-Hapax) ships as one PureScript module + one Erlang `gen_server`. No substrate changes.
7. The McMullen Yellow column played through Vetula → cv-router → modular polyphony, simultaneously echoing the chord roots through SuperDirt as a sample-based piano, with an Odonus melody threading over the current voicing through a soft-synth in Ableton. This is the "everything connects" demo.
8. **The cross-cell test.** Four separate cells — `chord1`, `melody`, `drums`, `aux` — each its own hot-loadable module. Editing `chord1`'s voicing strategy recompiles only `chord1`; `melody` keeps its tick going and picks up the new voicing through the bus on its next read. No cascade recompile. This is the test that the §5 layer is actually working in practice.

If all eight hold, the architecture is doing its job.